Naruto 5e — Rules Specification (THE RULES)

One of three project documents (each one responsibility):

***THE RULES** (this) — what the game is · **THE ARCHITECTURE** (`naruto5e_architecture.md`) — how the system is built · **THE BUILD PLAN** (`naruto5e_build_plan.md`) — how to build it & what remains.*

>

*This document specifies the game itself: the 14-chapter Naruto 5e ruleset, the seven layered systems (attribution, visualizer, app shell, action economy, DM build mode, merit economy, jutsu point-model), and the content tools (jutsu-builder, freeform resolver). It defines **engine capabilities** — the mechanics the engine resolves. How those capabilities are exposed (engine/MCP/LLM tiers, the write/read law, the intent contract, batch, educational failures) lives in THE ARCHITECTURE.*

Naruto 5e — Rules Reference

*A tool surface for running the Naruto 5e conversion as a persistent, multiplayer-capable RPG engine, in the lineage of `mnehmos-rpg`. Built **chapter by chapter from the source rulebook** — every field below is grounded in the actual text, not anime memory.*

>

***Core philosophy (carried from prior design work):** the engine owns truth; tools mutate and query persistent state; dice are authoritative; players, NPCs, and the DM all submit intent into a shared adjudication surface. A "room" is a locus of attention. One writer per room.*

Confirmed global facts (from Ch. 1 + "What's Different")

• **Spells** → **Jutsu.** Cantrips → **E-Rank** (with a chakra cost). Jutsu are **Ranked E→S**, not leveled: E=cantrip, D=1st–2nd, C=3rd–4th, B=5th–6th, A=7th–8th, S=9th.

• **Casting split into three systems:** Ninjutsu, Taijutsu, Genjutsu (plus Bukijutsu/Hijutsu as classifications).

• **Two resource pools per character:** Hit Points (Hit Dice) **and** Chakra Points (Chakra Dice).

• **Proficiency bonus is +3 at level 1.**

• **XP is "Mission Points."**

• **Power tiers by level:** Genin 1–4, Ch■nin 5–8, J■nin 9–12, Kage 13–15, Legendary 16–20.

• **28 clans** confirmed by extraction: Aburame, Akimichi, Bakuton, Fuma, Futton, Hatake, Hebi, Hoshigaki, Hozuki, Inuzuka, Jiton, Kaguya, Kurama, Kuru, Namikaze, Nara, Ranton, Ryu, Sarutobi, Senju, Shakuton, Shikigami, Tsuchigumo, Uchiha, Uzumaki, Yamanaka, Yoton, Yuki. (Plus "non-clan".)

• **708 jutsu** total across ranks (E:29, D:227, C:181, B:134, A:91, S:46).

CHAPTER 1 → `character_manage`

The character-creation and progression engine. Mirrors the book's 7-step build.

Persistent character state (the record this tool owns)

```
Character {
  id, name, ownerId, roomId
  clan            // one of the 28 + "none"; grants ability increases, traits, clan-jutsu access
  class           // Genjutsu Specialist | Hunter-Nin | Scout-Nin | Taijutsu Specialist | Summoner | ...
  background      // Ambition/background; grants an ability increase + proficiencies
  level           // 1..20
  rank            // Academy | Genin | Chunin | Jonin | Kage | Legendary  (derived from level band, can be overridden)
  missionPoints   // XP

  abilities { str, dex, con, int, wis, cha }     // base, BEFORE clan/background increases
  abilityBonuses { ... }                          // applied clan + background increases (tracked separately for audit

  hp   { current, max, tempHp }
  chakra { current, max }                         // the second pool — first-class
  hitDice    { type, total, remaining }           // e.g. d8
  chakraDice { type, total, remaining }           // die type set by class

  proficiencyBonus    // +3 at L1, scales by level
  ac, speed
  proficiencies { armor[], weapons[], tools[], skills[], savingThrows[] }
  clanTraits []       // e.g. Sharingan, Byakugan, Creepy Crawly, Cold to the Bone
  classFeatures []
  conditions []
  jutsuKnown []       // refs into jutsu_manage; capped by class "Jutsu Known"
  equipment []        // refs into equipment
  ryo                 // currency
}
```

Operations

shell, 0 mission points		
bility increases, traits, proficiencies, unlock clan		
Chakra Die type, casting stat, L1 features, , Jutsu Known cap		
ound ability increase + proficiencies		
andard_array |	roll_4d6 |	point_buy |
ived: HP/Chakra max, AC, prof bonus, mod table; et		
record (the read path)		
utation (DM authority)		
nges; handles temp HP, 0-HP/downed		
ool (called by jutsu_manage on cast)		
	long; spend hit/chakra dice to recover both pools	
ags canLevelUp at thresholds		
Hit/Chakra die, features, ASI/feat, Jutsu Known		
ng		

Adjudication notes (engine-vs-DM boundary)

- **Pure code (no LLM):** ability-score math, modifiers, HP/Chakra/dice computation, prof bonus, rank-from-level, point-buy validation, rest recovery. All deterministic.
- **DM/LLM only:** narrating creation, ruling on mixed-clan or homebrew edge cases, approving `manual` scores.

Enums grounded in source

- **clan:** the 28 listed above + `none`.
- **ability method:** `standard_array(15,14,13,12,10,8)`, `roll_4d6`, `point_buy(27)`, `manual`.
- **rank↔level band:** Genin 1–4 / Ch■nin 5–8 / J■nin 9–12 / Kage 13–15 / Legendary 16–20.

TODO (resolve in later chapters)

- `class` enum + per-class Hit/Chakra die, casting stat, Jutsu Known table → **Chapter 4**.
- `background` enum + bonuses → **Chapter 3**.
- `clanTraits` exact mechanics per clan → **Chapter 2**.
- `jutsuKnown` cap formula → **Chapter 4 + 9**.

CHAPTER 2 → Clan definitions (feeds `character_manage.set_clan`)

Chapter 2 defines the playable clans. **Correction from earlier extraction:** the document has **18 playable clans in Chapter 2** — an earlier loose scan wrongly counted 28 by sweeping in elemental-release labels and adversary clans from Chapter 14 (Bakuton, Futton, Jiton, Ranton, Shakuton, Yoton, Hozuki, Shikigami, Namikaze, Senju). Those are **NPC/adversary clans**, not PC options. The real PC roster is the 18 below.

Per-clan data shape (the structure every clan entry follows)

```
Clan {
  name
  signatureTrait        // named flavor trait (e.g. "Creepy Crawly", "The Curse of Hatred")
  abilityIncrease       // e.g. {int:+2, wis:+1}  (some offer a choice)
  speed                 // base walking speed (commonly 30 ft)
  skillProficiencies[]  // two clan skills (some are "pick from a list")
  features[]            // mechanical clan features (incl. clan-jutsu access + unique systems)
  clanJutsu { D[], C[], B[], A[] }   // a private jutsu tree; cataloged in Ch.10-12 pass
}
```

Every clan grants **access to its own clan-jutsu list** (added to your jutsu options) plus, often, a **unique resource/mechanic** (e.g. Akimichi *Calories*; Uchiha *Sharingan*).

The 18 playable clans (verified, with ability increase + signature trait)

Clan	Ability Increase	Signature Trait	Notable skills
Aburame	+2 INT, +1 WIS	Creepy Crawly (insect symbiosis)	Nature, Animal Handling

Akimichi	+2 CON, +1 STR	Big Appetite (Calories resource, body-size jutsu)	Athletics, Survival
Fuma	+2 DEX, +1 WIS	We Never Miss (thrown/wire weapons)	Perception, Martial Arts
Hatake	+2 INT, +1 CHA	Baring White Fangs	Ninshou, Perception
Hebi	+2 STR, +1 CON	(snake-based; Adaptive Camouflage)	Survival, Animal Handling
Hoshigaki	+2 CON, +1 STR	The Tailless Beast (huge chakra)	Animal Handling, Athletics
Inuzuka	+2 WIS, +1 STR or DEX	The Most Loyal (ninken companion)	Animal Handling, Acrobatics
Kaguya	+2 STR or DEX, +1 (choice)	Savage Battle Instincts (bone jutsu)	Athletics, Martial Arts
Kurama	+2 WIS or CHA, +1 INT	Illusions as Real as Reality	Illusions, Insight
Kuru	+2 WIS, +1 CON	(Dark Whip techniques)	Insight, Chakra Control
Nara	+2 INT, +1 CHA	Inconceivable Foresight (shadow jutsu)	Investigation, Insight
Ryu	+2 INT, +1 CON	Wrath of a Dragon	Ninshou, Chakra Control
Sarutobi	+2 STR, +1 CON	Bound by a Code of Honor	(pick any two)
Tsuchigumo	+2 WIS, +1 DEX	Arachnophobia (spider/thread jutsu)	Acrobatics, Chakra Control
Uchiha	(choice) — TODO exact line	The Curse of Hatred (**Sharingan** d■jutsu)	Insight, Ninshou/choice
Uzumaki	+2 CON, +1 CHA	Never Backing Down (huge HP, sealing)	Chakra Control, Ninshou
Yamanaka	+2 CHA, +1 INT	Impossible to Pin Down (mind-transfer)	Illusions, Persuasion
Yuki	+2 DEX, +1 INT	Cold to the Bone (Ice Release)	Chakra Control, Ninshou

How this plugs into the engine

`character_manage.set_clan(id, clan)` now resolves against this table: apply the ability increase, set base speed, grant the two skill proficiencies, attach the signature trait + any unique resource/mechanic, and unlock the clan's D→A jutsu tree as selectable options. Adversary/elemental clans (Ch.14) are handled separately by `adversary_manage`, **not** offered to PCs here.

TODO

- Exact Uchiha ability-increase wording + Sharingan progression mechanics (re-read p56-59).
- Unique clan resources to model as first-class state: Akimichi *Calories*, Uzumaki HP bonus, Inuzuka *ninken* companion, d■jutsu (Sharingan/Byakugan) activation states.
- Clan-jutsu tree contents → populated in the Ch.10-12 jutsu-extraction pass.

CHAPTER 3 → Backgrounds & the Will of Fire

Chapter 3 covers character *identity* — Ambitions/Drive/Goals/Fears (roleplay prompts), the

Will of Fire resource, and the **10 Backgrounds** (each granting proficiencies + an ability increase + a Feature). Two engine-relevant systems here: `background` data, and a new tracked resource, `willOfFire`.

3a. Will of Fire — a tracked table resource (this system's "Inspiration", upgraded)

```
WillOfFire {
  holder            // characterId
  has               // boolean — you either have it or you don't (NOT stackable)
}
```

Rules (from p69):

- Every player **begins each mission with a Will of Fire.**
- The GM awards it for playing to your Ambition/Drive/Goals/Fears or otherwise compelling play.
- **Not stockpilable** — you have one or you don't.
- **Spend it for any one of:** advantage *or* +5 on an attack roll; advantage *or* +5 on a save; advantage *or* +5 on an ability check; slightly alter the scene in your favor; gain an immediate reaction (even for a standard-action ability/jutsu); gain insight/a clue; **auto-succeed a death saving throw.**
- **Giftable:** a holder may give up their Will of Fire to award it to another player for great play.

Engine ops (`willoffire_manage` or folded into `character_manage`):

`grant(charId)`, `spend(charId, use)`, `gift(fromId, toId)`, `reset_on_mission_start(roomId)`.

This is a *table-covenant* mechanic — it rewards roleplay and can be passed between players, so it lives at the session layer, refreshed per mission.

3b. Backgrounds — feeds `character_manage.set_background`

```
Background {
  name
  skillProficiencies[]
  toolProficiencies[]
  startingEquipment[]
  equipmentPack          // choice
  feature { name, description }    // a roleplay/utility benefit
  abilityIncrease { name, choice }  // a NAMED +1, usually "+1 to one of two abilities"
}
```

The 10 backgrounds (verified structure; ability increase is a named +1 choice)

Background	Skills	Feature	Ability Increase (+1 to choice)
Entertainer	Acrobatics, Performance	Back by Popular Demand	"Enticing Personality": CHA or INT
Genius	choose two (Ninshou, Martial Arts, …)	Meeting Expectations	"League of Your Own": WIS or INT
Hard Worker	Acrobatics, Athletics	Focus and Grit	"Amazing Discipline": STR or DEX
Hermit	(TODO read p70)	(TODO)	(TODO)

Leader	History + (Deception or Persuasion)	Leadership Presence	"Captain Commander": (TODO ability)
Noble	(TODO read p70)	(TODO)	(TODO)
Student	(TODO read p70)	(TODO)	(TODO)
Traveler	Insight, Perception	The Exotic Individual	"Experienced": (TODO ability)
Trouble Maker	Deception, Sleight of Hand	Pitiable	(TODO ability)
Urchin	(TODO read p70)	(TODO)	(TODO)

(Confirmed pattern: each background = skill profs + tool profs + starting equipment + an equipment-pack choice + a named Feature + a named +1 ability increase offering a choice of two abilities. The TODO cells are entries whose text columns interleaved in extraction — values to be finalized on a targeted re-read of p70, not new structure.)

How this plugs into the engine

`character_manage.set_background(id, background)` applies: skill + tool proficiencies, starting equipment + chosen pack, the named Feature, and the +1 ability increase (player picks within the allowed pair). `willOfFire` is initialized when a mission starts and refreshed each mission.

TODO

- Finalize the 4 interleaved background rows (Hermit, Noble, Student, Urchin) + the two pending ability choices (Leader, Traveler, Trouble Maker) via targeted re-read of p70.
- Decide home for `willOfFire`: session/room layer vs character record (leaning session, since it resets per mission and can be gifted between players).

CHAPTER 4 → Classes (feeds `character_manage.set_class`)

The biggest mechanical chapter. **8 classes**, each setting Hit Die, Chakra Die, proficiencies, and a unique feature set. The defining design pattern: **Hit Die and Chakra Die are inversely related** — casters are fragile but chakra-rich; the taijutsu bruiser is tanky but chakra-poor.

Per-class core table (verified from the source)

Class	Hit Die	Chakra Die	Saving Throws	Archetype
Genjutsu Specialist	d6	**d12**	CON, CHA	illusion caster (fragile, huge chakra)
Ninjutsu Specialist	d6	**d12**	(caster pair)	elemental/ninjutsu caster
Intelligence Operative	d8	d10	DEX, INT	info/control specialist
Medical-Nin	d8	d10	(support pair)	healer/support
Scout-Nin	d8	d10	(skirmish pair)	recon/skirmisher
Hunter-Nin	d10	d8	(martial pair)	tracker/assassin
Weapon Specialist	d10	d8	STR/DEX	bukijutsu/weapons

Taijutsu Specialist	**d12**	d6	STR, CON	body/martial bruiser (tanky, low chakra)

(Save pairs confirmed in the source: Constitution+Charisma, Dexterity+Intelligence, Strength+Constitution, Strength+Dexterity — distributed across the 8 classes.)

Casting ability is keyed to JUTSU TYPE, not the class (key schema rule)

Confirmed across all classes:

- **Ninjutsu** attack/DC uses **Intelligence**.
- **Genjutsu** attack/DC uses **Wisdom**.
- **Taijutsu** keys off **Strength/Dexterity** (physical).

So a character can have **three different casting modifiers at once**, one per jutsu type. The engine computes jutsu attack = prof bonus + (type's ability mod); save DC = 8 + prof + (type's mod). This means `jutsu_manage.cast` must look up the casting ability **from the jutsu's type**, not from a single "spellcasting stat" on the class. (Big divergence from vanilla 5e — there is no one spellcasting ability.)

What `set_class` must populate

```
ClassDef {
  name
  hitDie            // d6..d12
  chakraDie         // d6..d12 (inverse of hitDie tendency)
  savingThrows[]    // two abilities
  proficiencies { armor:["Light"], weapons:["Simple"|...], tools[], skills{fixed[], chooseN, from[]} }
  jutsuKnown        // progression by level (per-class table) — TODO extract full tables
  features[]        // class features by level, incl. subclass choice (e.g. Genjutsu Pledge,
                    //   Scouting Technique, Taijutsu Style)
  subclassChoiceLevel
}
```

Subclasses (each class has a sub-path chosen at a set level)

Confirmed examples from the TOC/text: Genjutsu Specialist → **Genjutsu Pledges** (Illusionist…); Scout-Nin → **Scouting Technique** (Assault / Defensive / Tactical / Cloning Scout) + Maneuvers; Taijutsu Specialist → **Taijutsu Style** (Passionate Youth, Weapon Specialist…). Summoner appears as a feature/path (**Efficient Molding**, summoning). Full per-subclass features → later targeted read.

How this plugs into the engine

`set_class(id, class)` sets Hit/Chakra die (→ recompute both pools), the two save proficiencies, armor/weapon/tool/skill proficiencies, the Jutsu Known cap, and L1 features. `level_up` consults the per-class feature + Jutsu-Known progression. `jutsu_manage.cast` derives the casting modifier from the **jutsu type** (Nin=INT, Gen=WIS, Tai=STR/DEX).

TODO

- Full per-class **Jutsu Known progression tables** (by level) — needed for the Jutsu Known cap.
- Full **feature-by-level lists** and **subclass** feature sets for all 8 classes.

• Confirm exact save pairs per individual class (4 pairs are known; map each to its class).

CHAPTER 5 → Equipment & Inventory (`equipment_manage`)

Concrete gear chapter → the engine's inventory system. Currency is **Ryo**; starting wealth is **by class** (plus the wallet your background grants). Items span weapons, ninja tools, armor, consumables, and general gear, each priced in Ryo.

Item record

```
Item {
  id, name, type            // weapon | ninja_tool | armor | consumable | gear
  valueRyo
  // weapons:
  damage { dice, type }     // e.g. 1d6 Piercing, 1d8 Slashing, 2d6 Slashing
  properties[]              // thrown, finesse, two-handed, reach, etc.
  proficiencyClass          // simple | martial / ninja-tool
  // armor:
  acBase, acRule, movementPenalty   // Flak Jacket -> Battle Armor -> Shinobi Battle Armor (tiers)
  // consumables (the Naruto-flavored part):
  effect { onUse }          // e.g. soldier/blood pills restore chakra; smoke bomb obscures; tag explodes
  charges
}
```

Confirmed sample data (grounding the schema)

• **Weapons (name / cost / damage):** Sling 1 Ryo (1d6 P?), Hand Axe 5, Hand Scythe 5, Quarterstaff 5, Weighted Chain 5, Kunai 10, Chained Hand Scythe 10, Senbon 15, Broadsword 15 (1d8 Slashing), Combat Bracers 15, Jitte 15, Nunchaku 15, Chained Spear 20, Short Bow 25, Shuriken 25, Iron Claw 25, Hooked Lance 30, Great Axe 40 (2d6 Slashing), Tetsubo 40, Naginata 45, Scythe 45. Damage types seen: Piercing, Slashing (and dice up to 2d6, 1d10).

• **Armor tiers:** Flak Jacket → Armored Flak Jacket → Battle Armor → Shinobi Battle Armor (increasing protection, increasing movement restriction).

• **Ninja tools:** kunai, shuriken, senbon, smoke bombs, wire, etc. — many are *thrown weapons* and/or *consumables with effects*.

• **Currency:** Ryo (interchangeable across most countries; Land of Iron / Land of Moon have own mints).

• **Equipment Packs:** named starting bundles chosen at creation (e.g. Diplomat's, Entertainment).

Operations (`equipment_manage`)

op	params	effect
`give` / `remove`	charId, item	add/remove from inventory
`equip` / `unequip`	charId, itemId	weapons/armor → recompute AC & attacks on the sheet
`use_consumable`	charId, itemId	spend a charge, apply effect (e.g. restore chakra)
`buy` / `sell`	charId, itemId, price	adjust Ryo, transfer item
`grant_starting_wealth`	charId	by class + background wallet
`choose_pack`	charId, pack	apply a named equipment pack at creation

Adjudication

Pure code: Ryo math, carrying/proficiency checks, AC recompute from armor, consumable charge tracking, weapon damage dice. DM only narrates acquisition or rules on exotic/improvised items.

TODO

- Full weapon table with **properties per weapon** and complete damage entries.
- Armor AC values + movement penalties per tier.
- Consumable effect text (soldier pills, tags, smoke/flash bombs) → model `effect.onUse`.
- Starting-wealth-by-class amounts + equipment-pack contents.

CHAPTER 6 → Using Ability Scores (skills, checks, saves — mostly engine math)

Largely deterministic: d20 + ability mod + (proficiency if trained) vs DC. The engine resolves all of it in code. The one thing that *must* be locked is the **custom skill list**, because clans, classes, and backgrounds all grant skill proficiencies that must point at real skills.

The complete skill list, by governing ability (verified, p139–143)

Ability	Skills
Strength	Athletics, **Martial Arts**
Dexterity	Acrobatics, Sleight of Hand, Stealth
Constitution	**Chakra Control** *(the only CON skill — unique to this system)*
Intelligence	**Crafting**, History, Investigation, Nature, **Ninshou**
Wisdom	Animal Handling, **Illusions**, Insight, Medicine, Perception, Survival
Charisma	Deception, Intimidation, Performance, Persuasion

Naruto-specific additions (not in vanilla 5e): **Martial Arts** (STR), **Chakra Control** (CON), **Crafting** & **Ninshou** (INT), **Illusions** (WIS). These are exactly the skills seen granted by clans/backgrounds earlier — now they resolve to real entries.

Engine notes

- `skill_check(charId, skill)` → d20 + abilityMod(skill→ability) + (profBonus if proficient).
- Saving throws: d20 + abilityMod + (profBonus if the class grants that save).
- Advantage/disadvantage, passive scores (10 + mods), and group checks all standard.
- This chapter adds **no new tool family** — it confirms the skill enum used across `character_manage` (proficiencies), `combat`, and jutsu casting. All pure code; DM sets DCs / calls for checks.

Resolved upstream

The clan/background skill grants from Ch.2–3 now validate against this list (e.g. Aburame's

"Nature, Animal Handling", Yuki's "Chakra Control, Ninshou", Yamanaka's "Illusions, Persuasion" all map correctly).

CHAPTER 7 → Adventuring, Missions & Downtime (`mission_manage` + rest/downtime)

The play-loop chapter — the proto-MMO heartbeat. Two engine systems: the **mission board** (the grind/rank-up loop) and **downtime** (how a genin gets strong between missions). Plus resting, which must handle **both pools**.

7a. Missions — ranked D→S, gated by ninja rank

Confirmed: mission ranks parallel the technique ranks and map to ninja rank.

- **D-Rank** — genin; trivial tasks (find pets, weed gardens). Low pay, ~no danger.
- **C-Rank** — chunin (sometimes genin); bodyguard duty, hunting beasts; real risk.
- **B-Rank** — jonin/chunin; spying, assassination; expect enemy ninja.
- **A-Rank** — elite; high danger.
- **S-Rank** — legendary; deadliest.

```
Mission {
  id, title, rank            // D..S
  requiredRank               // min ninja rank to accept
  rewardRyo, rewardMissionPoints
  status                     // posted | accepted | active | resolved | failed
  assignedTo []              // characterIds (a squad)
  roomId / locale
}
```

Ops (`mission_manage`): `post`, `list_board`, `accept`, `resolve(outcome)` (pays Ryo + mission points, may raise rank), `fail`, `rank_up(charId)`. This is the loop that drives progression and ties directly into `character_manage.add_mission_points`.

7b. Resting — dual-pool recovery (critical for this system)

`character_manage.rest(charId, type)`:

- **Short rest:** spend **Hit Dice** to recover HP *and* **Chakra Dice** to recover Chakra (roll die + CON mod each).
- **Long rest:** restore pools per the book's long-rest rules; recover a portion of spent dice.

The engine tracks `remaining` for both dice pools; this is why both were first-class from Ch.1.

7c. Downtime activities (verified mechanics) — the "get strong" layer

Activity	Time	Cost	Effect
Training	25 weeks	50 Ryo/week	with an instructor: learn a new language, tool proficiency, **weapon**, or **Feat**
Researching	GM-set weeks	(varies)	uncover campaign info/mysteries if available

Recuperating	1 week	—	DC 15 CON save → end an effect blocking HP regain, or advantage vs a disease/poison for a week
Shopping	1 week	—	find your target items at a **5d4% price reduction**

Ops (`downtime_manage`): `train(charId, target)`, `research(charId, topic)`, `recuperate(charId)`, `shop(charId, items)`. Training is the long-haul self-improvement track (feats/weapons/tools); note **learning new jutsu** is governed by class Jutsu-Known + level-up (Ch.4/9), while training covers feats/proficiencies.

Adjudication

Pure code: Ryo/mission-point math, rank thresholds, rest dice recovery, the recuperate save, the 5d4% shopping discount, downtime time/cost bookkeeping. DM authors mission content, gates research, and provides instructors — i.e. the *fiction* of the loop, not the math.

TODO

- Exact mission **reward tables** (Ryo + mission points by rank).
- Long-rest dice-recovery fraction (confirm exact rule on p185–188).
- Travel/pace/exploration specifics (standard; low priority).

CHAPTER 8 → Combat (`combat_manage` + `combat_action`)

The fight engine. Mostly the standard 5e combat skeleton (deterministic, code-resolved), with Naruto-specific seams: **Cast a Jutsu** spends chakra, and **Clashing Jutsu** lets techniques collide. Initiative is the engine's **turn authority** — the thing that decides whose intent resolves when (ties directly to the multiplayer room/turn model).

Encounter state

```
Encounter {
  id, roomId
  combatants [ { actorId, initiative, isPC, team } ]
  order []                  // sorted by initiative — the turn authority
  round, activeIndex
  status                    // active | ended
}
```

Turn structure & action economy (confirmed)

- **Initiative** at start; act in order; rounds.
- Each turn: **move + one action + (optional) one bonus action**, plus **reactions** (one per round).
- **Actions:** Attack, **Cast a Jutsu**, Dash, Disengage, Dodge, Help, Hide, Ready, Search, Use an Object.

The Naruto seams

• **Cast a Jutsu (the combat↔jutsu hook):** a jutsu's *casting time* may be an action, a **reaction**, or longer — so casting is "not necessarily an action." `combat_action(cast)` must: read the jutsu's casting time, **check & deduct chakra cost** (via `character_manage.spend_chakra`), then resolve the jutsu effect (attack roll / save / condition / damage). This is where Ch.8 and Ch.9 meet.

• **Clashing Jutsu:** when two jutsu collide, the engine adjudicates the clash (opposed check/contest) — a setpiece Naruto mechanic. (Exact resolution rule → targeted read.)

• Reactions can be jutsu (e.g. substitution-style dodges) — the engine supports reaction-cost jutsu.

Attacks, damage, conditions

• **Attack roll** = d20 + ability mod + prof (if proficient) vs **AC**; crits on 20.

• Jutsu attack/DC uses the **type-based** casting ability (Nin=INT, Gen=WIS, Tai=STR/DEX) — Ch.4 rule.

• **Damage** applies to the HP pool; resistance/vulnerability/temp HP handled.

• **Conditions (full list present):** Blinded, Charmed, Deafened, Frightened, Grappled, Incapacitated, Invisible, Paralyzed, Petrified, Poisoned, Prone, Restrained, Stunned, Unconscious, plus **Exhaustion**. Jutsu impose/remove these.

• **Dropping to 0 HP:** downed → **death saving throws**; recall a **Will of Fire** can auto-succeed a death save (Ch.3). Knockout (non-lethal) supported.

`combat_action` operations

`attack`, `cast` (chakra-checked), `dash`, `disengage`, `dodge`, `help`, `hide`, `ready`, `search`, `use_object`, `bonus_action`, `reaction`. `combat_manage`: `start` (roll initiative), `advance` (next turn — the turn authority; on an agent's turn with autoOnTurn, fires its intent), `add/remove combatant, end`.

Adjudication

Pure code: initiative order, attack/damage/save math, chakra deduction, condition tracking, death saves, turn advancement. DM/agent: declares *intent* (what a combatant attempts); the engine resolves it. NPC combatants are agents submitting intent into the same `combat_action` surface as players — the symmetry from the architecture work.

TODO

• Exact **Clashing Jutsu** resolution rule (p180).

• Confirm bonus-action/reaction jutsu economy specifics.

• Cover/flanking/range bands if the conversion modifies them.

CHAPTER 9 → Jutsu Casting (`jutsu_manage`) — THE KEYSTONE

The soul of the system. Defines what a jutsu *is* and exactly how casting resolves. Every jutsu in Ch.10–12 (708 of them) is an instance of the definition below; this chapter is the envelope they

pour into.

Jutsu definition (the verified stat-block anatomy)

```
Jutsu {
  name
  classification   // Ninjutsu | Taijutsu | Genjutsu | Bukijutsu | Hijutsu (+ Medical etc. as keywords)
  rank             // E | D | C | B | A | S   (E = cantrip-tier but STILL costs chakra)
  prerequisites    // optional (e.g. a clan, a feature)
  castingTime      // 1 action | 1 reaction | bonus | minutes | hours  (NOT always an action)
  range            // self | touch | feet | "a point in space" | creature-target
  duration         // instant | rounds | minutes | hours | years | "until dispelled" | Concentration
  components []    // HS=hand seals, CM=chakra molding, CS=chakra seals, M=mobility, W=weapons, NT=ninja tools
  cost             // CHAKRA points
  keywords []      // e.g. Hijutsu (clan-only), Medical (needs feature/feat), element releases
  description      // effect text (the DM-adjudicated / engine-structured part)
  atHigherRanks    // upcast rule: extra chakra → scaled effect (e.g. "+3 cost, +2d4 dmg per rank")
}
```

Casting rules (verified, p188–192)

• **Jutsu Known:** you can only cast jutsu you've learned; how many you may *know* is capped by a per-class, per-level table (Ch.4 TODO). Learned jutsu are always "prepared."

• **Chakra cost:** every cast spends chakra from the pool (even E-rank). Higher rank = more taxing. Run out of chakra → can't cast. Recovered by resting (Ch.7).

• **Casting time:** action / reaction / bonus / longer — read per jutsu; reaction-cost jutsu enable substitution-style defensive plays.

• **Components:** must be able to provide HS / CM / CS / M / W / NT or you can't cast. (Engine can check: e.g. bound hands block HS; silenced doesn't matter, but restrained may block M.)

• **Casting at a Higher Rank (upcast):** spend more chakra to cast a known jutsu at a higher rank, scaling the effect per its `atHigherRanks` clause. The engine reads the clause and applies the delta.

• **Concentration:** sustained jutsu require concentration — and notably **you may hold up to TWO concentration jutsu at once** (divergence from vanilla 5e's one). Engine tracks ≤2 concentration slots.

• **Attack / Save:** uses the **type-based casting ability** (Nin=INT, Gen=WIS, Tai=STR/DEX). jutsu attack = prof + typeMod; save DC = 8 + prof + typeMod.

• **Keywords gate access:** *Hijutsu* = clan-only; *Medical* = requires a feature/feat to learn; element releases (Fire/Water/Wind/etc.) tag affinities.

`jutsu_manage` operations

op	params	effect
`learn`	charId, jutsu	add to known (validate vs Jutsu Known cap, prereqs, keyword gates)
`forget` / `swap`	charId, jutsu	level-up swap rules
`list_known`	charId	known jutsu, filterable by type/rank
`cast`	charId, jutsu, [targets], [atRank]	**check+deduct chakra** → **resolve casting time** → **roll attack/save w/ type ability** → **apply effect**; if `atRank` > base, apply upcast
`check_castable`	charId, jutsu	components available? enough chakra? known?

`concentration_start / _end`	charId, jutsu	manage the ≤2 concentration slots
`define`	jutsu fields	register a jutsu into the catalog (used by Ch.10–12 import)

Adjudication (the parse/resolve/escalate split, applied to jutsu)

• **Pure code:** chakra check/deduct, attack/save math, upcast scaling via `atHigherRanks`, concentration-slot tracking, component availability, range/duration bookkeeping.

• **DM/LLM only:** narrating the technique; ruling on a novel/freeform use or an ambiguous interaction (e.g. a Clashing Jutsu contest, creative environmental use). This is exactly the "engine resolves the structured 90%, escalate the exotic 10%" boundary from the architecture work — the jutsu envelope is the structured part; the `description` is where escalation can happen.

TODO

- Per-class **Jutsu Known progression tables** (caps `learn`).
- Exact **upcast** parsing across all 708 jutsu (the `atHigherRanks` clause is free-text → normalize).
- Element-release affinity rules (Fire/Water/Wind/Earth/Lightning + advanced releases) — likely Ch.13/14.

CHAPTERS 10–12 → The Jutsu Lists (the 708-technique data layer)

The largest content layer: every jutsu poured into the Ch.9 `Jutsu` envelope. These chapters are **data, not new mechanics** — they populate `jutsu_manage`'s catalog via `define`.

Extraction status (programmatic parse of the source)

The stat blocks are uniform, so a code parser walks the document and extracts each block's fields. Honest results of the automated pass:

Field	Coverage	Notes
Blocks located	**674 / 708** (~95%)	remainder split across page breaks / OCR seams
Chakra cost	**659** parsed	clean integer costs
Rank (E–S)	**670** parsed	D:215 C:173 B:127 A:85 E:27 S:43 (+4 unresolved)
Classification	~674	Ninjutsu/Taijutsu/Genjutsu/Bukijutsu/Hijutsu
Casting time / range / duration / components	high	per-block fields
Upcast (`atHigherRanks`) present	**257** flagged	the rest are fixed-effect
Name	**627** recovered (was ~436)	after the recovery pass — see note — names that span lines / cross page breaks need cleanup

The parsed catalog is saved as structured data (`jutsu_parsed.json`) — 674 records, each with classification, rank, cost, casting time, range, duration, components, and an upcast flag.

Honest assessment (important)

• **Mechanical fields extract reliably (~95%)** — rank, cost, classification, components, upcast. These are what the engine needs to *resolve* a cast, and they're in good shape.

• **Names are the lossy field (~65% clean).** The PDF's two-column layout + page breaks split many technique names from their blocks. Pushing the parser harder would risk *wrong* names, which is worse than blank ones — so the remaining names are flagged for **hand-verification against the source**, not fabricated. (Consistent with the project rule: never invent data.)

• This is the realistic shape of bulk-importing a 381-page fan PDF: structure transfers, labels need a cleanup pass. A second pass with the per-clan/per-class jutsu sub-lists (which list names in a cleaner context) will recover most missing names.

How it plugs in

Each record → `jutsu_manage.define(...)` registers it in the catalog. `learn`/`cast` then reference catalog entries. Clan jutsu trees (Ch.2) and class jutsu (Ch.4) are **filtered views** of this same catalog (by keyword/classification/clan), not separate data.

TODO

• **Name-recovery pass:** re-parse the per-clan and per-class jutsu sub-sections (cleaner name context) to fill the ~270 missing/uncertain names; reconcile to the full 708.

• Normalize the free-text `atHigherRanks` clauses into structured scaling rules (cost delta + effect delta) for engine auto-upcast.

• Tag element releases (Fire/Water/Wind/Earth/Lightning + advanced) from `keywords`.

CHAPTER 13 → Customization (multiclassing + feats)

Optional advanced builds. Two systems, both feeding `character_manage` (level-up choices).

Multiclassing

• Gain a level in a **new class** instead of your current one when you level; levels across all classes **sum to character level** (e.g. Scout-Nin 3 + Intelligence Operative 2 = level 5).

• **Prerequisites** apply (minimum ability scores to enter/leave a class — standard 5e gate).

• Special combining rules for casters: **"Jutsu Known & Highest Rank Known"** governs how a multiclassed ninja's technique access stacks across classes (so a Nin/Gen multiclass tracks each casting track properly) + proficiency and fighting-style stacking rules.

Engine: `character_manage.level_up(charId, {intoClass})` supports choosing a *different* class; the sheet tracks `classes[] = [{class, level}]`, derives total level, and recomputes pools, prof bonus, and Jutsu-Known from the combined progression.

Feats

- Taken **in place of an Ability Score Improvement** at the levels a class grants one.
- Large catalog with Naruto-flavored entries. Sample confirmed: Acrobat, Actor, Advanced Study, Agile Feint, Alchemist, Alert, Animal Handler, Apex Heritage, Athlete, Blinding Agility, Bloodthirsty, **Branch Family Training**, Brawny, **Bukijutsu Archivist**, … (full list to import).

```
Feat {
  name
  prerequisite       // optional (ability/clan/level gate)
  effects[]          // ASI grants, proficiencies, special abilities, jutsu access
}
```

Engine: `feat_manage.list`, `character_manage.take_feat(charId, feat)` (validates prereqs; applies effects), offered as the ASI-or-feat choice during `level_up`.

Adjudication

Pure code: multiclass level math, prereq validation, pool/prof/Jutsu-Known recomputation, feat prereq checks + effect application. DM only approves unusual concept combos.

TODO

- Full **feat catalog** import (names + prereqs + effects) — a content-extraction pass like the jutsu list.
- Exact multiclass **Jutsu-Known/Highest-Rank** combining formula (p314).
- Multiclass proficiency-grant table (what you get entering a second class).

CHAPTER 14 → Allies & Adversaries (`adversary_manage`) — THE FINALE

The enemy engine + Bingo Book (premade foes). Adversaries are **not** built like PCs — they use a lighter 8-step template tuned for fast DM creation. This is also the home of the **elemental-release & NPC clans** (Bakuton, Futton, Jiton, Ranton, Shakuton, Yoton, Hozuki, Shikigami, Namikaze, Senju) correctly excluded from the PC roster in Ch.2.

NPC power baseline (from the source)

Civilians/guards rarely exceed level 4; ability 8 is average. Player characters are *extraordinary* and outgrow the population. Level bands: Genin 1–4, Ch■nin 5–8, J■nin 9–12, Kage 13–15, Legendary 16–20.

Adversary profile (trimmed character sheet)

```
Adversary {
  name, tier            // minion | elite | solo
  role                  // defender, striker, controller, etc.
  clan                  // optional — grants that clan's Hijutsu + adversary-clan features
  level / tierBand      // sets stat baseline
  ac, hp, speed
  abilities {}          // simplified
  traits[]              // 1–2 passive flavor/role traits (keep simple)
  roleAbilities[]       // role-specific powers (defenders defend, strikers strike)
  jutsu[]               // capped by Adversary Tier (rank limits to prevent burst-kills)
  attacks               // FREEFORM (see below)
}
```

The 8-step build (verified)

1. Concept/role → 2. tier (minion/elite/solo) → 3. **assign a clan** (opens clan Hijutsu + adversary-clan features) → 4. traits (1–2, keep simple) → 5. role-specific abilities → 6. assign jutsu (by role's types, **rank-capped by tier**) → 7. personalize (tune AC/HP/attack vs the party) → 8. play (improvise; refine mid-encounter).

The tiers (the scaling engine — the smart part)

- **Minion:** rank-and-file fodder; low HP, fought in groups; expendable.
- **Elite:** distinct, tougher threat — **Elite Actions** let it act multiple times per round.
- **Solo:** boss built to face a whole party alone — gains **Legendary Actions = (players – 1)**, so it scales to table size and gets enough turns to fight everyone.
- **Jutsu-rank cap by tier:** adversaries are limited in the jutsu ranks they can access, explicitly to **prevent sudden burst damage** that one-shots a player. Encounter balance baked into the tier.

Attacks: freeform + structured building blocks

- **Freeform attacks:** adversaries carry **no fixed attack list** — the DM describes an appropriate attack on the fly (speed + flexibility). The engine supports a freeform attack: attack bonus / save DC + damage scaled by tier/level, narrated ad hoc.
- Building blocks: **Freeform Jutsu, Condition Attacks/Jutsu, Area Attacks, Movement Attacks** — templated effect shapes the DM picks from. Plus concentration, healing jutsu, summoning rules.

`adversary_manage` operations

op	params	effect
`build`	concept, tier, role, [clan], level	spin up an adversary via the 8-step template
`set_tier`	id, tier	apply minion/elite/solo scaling (elite actions, solo legendary actions)
`assign_clan`	id, clan	grant clan Hijutsu + adversary-clan features (elemental clans live here)
`add_role_ability` / `add_trait`	id, …	role powers, 1–2 passive traits
`freeform_attack`	id, descriptor	tier/level-scaled attack, narrated
`from_bingo_book`	name	instantiate a premade foe (Anbu, Assassin, Bandit, Chunin, Genin, Haku, named characters…)

`scale`	id, partySize/level	the "Scaling Enemy" adjust noted throughout the Bingo Book

Bingo Book (premade foes)

Ready-made statblocks incl. generic ranks (Genin, Ch■nin, Anbu, Bandit, Assassin) and named characters, many with a **"Scaling Enemy"** variant to match party level. `from_bingo_book` loads these; `scale` tunes them.

How it ties the engine together

Adversaries are **agents that submit intent into the same** `combat_action` **surface as players** (Ch.8) — the symmetry from the architecture work. The DM/engine invokes an adversary on its initiative turn (`combat_manage.advance` → agent intent → resolve). Tiers decide how many actions it gets; the jutsu-rank cap keeps it fair. This is the tool that lets the world fight back.

TODO

- Import the **Bingo Book** statblocks (content pass like the jutsu list).
- Exact tier stat baselines (HP/AC/attack by level band) + elite-action counts.
- Adversary-clan feature variants for the elemental-release clans.
- Summoning + healing-jutsu adversary rules (p340).

Spec status (all 14 chapters)

Ch.1 character · Ch.2 clans (18 PC, correction logged) · Ch.3 backgrounds + Will of Fire · Ch.4 classes · Ch.5 equipment · Ch.6 skills · Ch.7 missions/downtime/rest · Ch.8 combat · Ch.9 jutsu casting (keystone) · Ch.10–12 jutsu catalog (674/708 parsed) · Ch.13 multiclass+feats · Ch.14 adversaries. **The full Naruto-5e MCP tool surface is specified end to end.**

FINAL PASS → Attribution, Content Separation & Local Deployment

This engine is a **non-commercial, personal, open-source project**, run locally and shared with trusted players via tunneling (e.g. Tailscale / Cloudflare Tunnel). Attribution and a clean engine/content split are first-class parts of the design — not afterthoughts.

Engine / content separation (architectural boundary)

• **Engine core (open, shareable):** the tool surface, adjudication engine, room-graph, multiplayer model, dice, state. This is *original work* and is what lives in the public repo. It is **IP-clean** — generic mechanics (dual resource pools, ranked abilities, tiered adversaries), no Naruto names, no copied text.

• **Content pack (local only):** the Naruto 5e data — clan write-ups, the parsed jutsu catalog, copied stat-block text, named characters. Loaded by the engine at runtime from a local pack the

user supplies; **not** committed to the public repo. "Bring your own copy of the document."

• The engine treats content as data behind a loader interface, so the same core could load *any* content pack. This keeps the repo clean and the IP local.

`attribution_manage` (built-in, always-on)

```
Attribution {
  ipCredit         // "Naruto © Masashi Kishimoto / Shueisha / Viz Media / Shonen Jump"
  conversionCredit // the fan-conversion author + playtesters (per the document credits)
  systemNotice     // 5e SRD / OGL or CC-BY notice text, verbatim, where the SRD applies
  disclaimer       // "Unofficial fan work. Not affiliated with or endorsed by the rights holders."
  commercialUse    // hard-coded false; engine runs in non-commercial mode only
}
```

• Every content entity (clan, jutsu, adversary, feat) carries a `source` field (document + page) so provenance travels with the data and content can never be surfaced stripped of its origin.

• `attribution_manage.get()` returns the manifest; the running instance can display its own credits.

• `commercialUse` defaults false and is **gated** — the engine will not run in any paid/subscription mode. (Decision: stay local & non-commercial.)

Local deployment model

• **Single host + tunnel** to trusted, fan players — not a public sign-up service.

• This *simplifies* the architecture: drop public multi-tenant auth, billing, abuse-prevention, and strangers-scale rate limiting. The room-graph / offscreen-tick / mega-room design all still work; they just scale *down* gracefully to a friend group.

• Railway/Postgres still fine as a private backend, or run entirely on the host machine + Postgres locally with the tunnel exposing the websocket to invited players.

This closes the spec: **all 14 chapters + attribution + deployment**, grounded in the source, honest about provenance, and scoped to a clean local/open-source project.

Ch.10–12 ADDENDUM → Name-recovery pass (completed)

A second parse using the *line-immediately-above-*`Classification` heuristic (names sit directly atop each stat block) recovered the lossy name field substantially:

• **627 of 674 parsed blocks now named (~93%)** — up from ~436. **89% of all 708 jutsu** named.

• **661** with chakra cost. Mechanical fields remain ~95%.

• **47 still unnamed** — names fused into preceding description text or broken across page seams.

Distribution: Ninjutsu 35, Hijutsu 10, Taijutsu 1, Bukijutsu 1. Left blank (not fabricated), flagged for manual verification against the source. Recovered catalog: `jutsu_recovered.json`.

3D TACTICAL COMBAT VISUALIZER

A **renderer of engine IR, never a source of truth.** The Ch.8 combat engine owns positions, HP, chakra, initiative, conditions, and emits structured events; the 3D layer subscribes and draws. Text-only players play the identical game — 3D is a *view*. Web-based (Three.js / React-Three-Fiber)

so it runs in a browser over the local tunnel with zero install.

Confirmed presentation choices

• **Camera:** locked **top-down** tactical, with **fit-to-zoom** (auto-frames all combatants; re-fits as the encounter spreads or shrinks). No orbit in v1 — clarity first.

• **Tokens:** simple **3D figures** (low-poly humanoid pawns) per combatant — readable from top-down, with team color, a facing indicator, and floating **HP + Chakra** bars (the dual pool, two bars).

• **Jutsu:** rendered as their **actual area shapes** — the geometry comes straight from the jutsu's `range`/area data (cone, sphere/radius, line, single-target marker), drawn on the grid when cast.

Scene model (every object is a projection of engine state)

```
Scene {
  grid          // measured tactical plane: 5e feet -> squares; terrain/elevation tiles
  tokens[]      // one per combatant: {pos, facing, team, hpBar, chakraBar, conditionRing, initBadge}
  activeGlow    // highlight on whose-turn-it-is (driven by combat_manage.advance)
  overlays[]    // movement range, attack/jutsu range, AoE templates, line-of-sight
  effects[]     // transient jutsu shapes + resolution flourishes
  turnRail      // initiative order UI
}
```

Render pipeline (IR → scene delta)

The visualizer is a **pure function from the combat event stream to scene changes** — zero combat logic.

```
{type:"move", actor, from, to}        -> tween figure across squares; re-fit camera
{type:"attack", actor, target, hit}   -> strike effect; target HP bar drops
{type:"cast", actor, jutsu, area}     -> draw the jutsu's AREA SHAPE (cone/sphere/line) from its range data;
                                         chakra bar drops by cost; element-keyed effect
{type:"condition", target, cond}      -> add/remove condition-ring icon
{type:"down", actor}                  -> figure falls; death-save pips appear
{type:"advance", actor}               -> move activeGlow + turnRail pointer
```

Same IR the text client consumes — this client just renders it in 3D.

Jutsu area shapes (from jutsu data, Ch.9)

The jutsu envelope's `range`/`duration` already encode the shape:

- `Self (20-foot sphere)` -> a 20ft radius sphere/disc centered on caster
- `60-foot line, 10 ft wide` -> a line template
- `30 feet radius` -> a ground disc
- `Range: 60 feet` + single creature -> a target marker + travel effect

The visualizer reads these and draws the matching template — so AoE is *literally the rules*, shown.

Element keyword (Fire/Water/Wind/Earth/Lightning/Ice...) tints the effect.

Interaction = intent submission, NOT resolution (the symmetry, made visual)

• Click a reachable square -> **submit move intent** to `combat_action`; engine adjudicates; the returned IR is what actually moves the figure. The client never moves a token on its own.

• Click a target / pick a jutsu -> **submit attack/cast intent**; engine resolves; IR drives the effect.

• Reachable squares, valid targets, and AoE previews are *hints* computed from engine-provided speed/range, but the commit always round-trips through the engine. The visualizer can only **propose**; the engine **disposes**. It cannot desync or cheat the world.

Diorama upgrade path (later, additive)

Because actions arrive as typed IR carrying the jutsu's `classification` + element keywords, the placeholder cast-effect can later be swapped for element-specific flourishes (fire/lightning particles, a hand-seal flash, figure animations) — pure polish on resolved events. The tactical layer never depends on it.

Build notes

- React-Three-Fiber scene; orthographic top-down camera; `fitBounds`-style auto-zoom to combatants.
- Subscribes to the room's combat IR over the websocket (the same stream from the engine).
- Optional entirely: a player can run text-only; the visualizer is a second renderer of one truth.

THE WEB APP SHELL (the layered aid around the visualizer)

The visualizer is the battlemap in the center; the app is everything around it. **Role-aware** (player + DM, same app), **balanced dashboard** layout. Organizing principle (constant with the whole engine): **every panel renders engine state; every control submits intent.** Role is a *permission + visibility filter*, enforced server-side — a player client cannot emit a DM intent.

Layout (balanced dashboard)

```
TOP BAR: village/scene · round+turn · your Will of Fire · role badge (Player|DM)
LEFT RAIL : Character Sheet (top) + Party panel (bottom)
CENTER    : Visualizer — 3D battlemap in combat; village/scene view out of combat
            + initiative rail + context Action Bar (move·attack·cast·use)
RIGHT RAIL: Chat / Narration feed (shared IR-as-prose + dice rolls)
BOTTOM DRAWER (tabs): Jutsu Browser · Inventory · Notes/Journal
```

The five panels (render-of-state + intent-emitter)

• **Character Sheet** — renders the engine character record: dual **HP/Chakra** bars, six abilities + the three casting mods (Nin=INT, Gen=WIS, Tai=STR/DEX), conditions, rank, mission points, Will of Fire. Controls *propose* (spend chakra, take damage, rest) → engine resolves. DM may open any sheet.

• **Party panel** — compact teammate cards (HP/chakra bars, condition icons, active-turn marker). Pure render of co-characters in the room. The "glance at my squad" view.

• **Chat / Narration feed** — the **one shared narrative stream per room** (D&D table model — not per-player). IR renders here as prose + inline dice rolls. DM posts narration; resolved actions auto-post.

• **Jutsu & Inventory browser** — known jutsu as cards (rank, chakra cost, range, AoE-shape icon);

gear. In combat, clicking a jutsu = **cast intent** → visualizer + engine. Filtered to known/owned.

• **Notes / Journal** — two scopes: **personal** (private) and **campaign canon** (shared, maps to `narrative_manage`: plot threads, NPCs, canonical moments). DM authors canon; players keep private notes.

Role-awareness

• **Player view:** own full sheet, party glance, shared narration, own jutsu/notes; intents scoped to their character; map hides DM-only fog/hidden tokens.

• **DM view:** DM control surface — open any sheet, see hidden enemy stats/HP, post narration, spawn adversaries (Ch.14), advance turns, reveal/hide map regions, author campaign canon.

• Enforced **server-side** by the engine; the UI merely hides disallowed controls.

Two center modes (one surface)

• **Out of combat:** village/scene view — location, who's present, exits; action bar = explore/talk/travel.

• **In combat:** the 3D tactical visualizer; action bar = move/attack/cast/use.

• Switching modes is just the engine starting/ending an encounter in that room.

Tech

React + the websocket subscription to the room's IR stream; the visualizer (R3F) docked center; panels are subscribers to slices of the same state. Runs in-browser over the local tunnel.

ACTION ECONOMY → `TurnBudget` (formalizes Ch.8 combat)

Ch.8 named the action economy; this defines how the engine **enforces** it. Each turn is a budget of typed resources; every `combat_action` declares a **cost descriptor** of what it spends. The engine unifies three gates — action economy, chakra cost, and components — into one deterministic "can you pay?" check before any dice roll.

The budget (reset at the start of each combatant's turn)

```
TurnBudget {
  action:    1
  bonus:     1
  reaction:  1          // refreshes at START of your turn; spendable BETWEEN turns
  movement:  speed      // a divisible pool (feet/squares), spent in pieces
  freeInteraction: 1    // the minor "draw a weapon / open a door"
}
```

Cost descriptor (every combat intent carries one)

A jutsu's descriptor derives from its `castingTime` (Ch.9):

• `1 action` → spends `action`

• `1 reaction` → spends `reaction`; legal only in response to a declared trigger

• `bonus` → spends `bonus`

- `minutes/hours` → NOT a combat action; routed as out-of-combat/ritual cast
- movement N → draws N from the `movement` pool (divisible: move, act, move again)

Resolution pipeline (pure code, pre-dice)

```
submitIntent(combatAction)
  → read cost descriptor (action? bonus? reaction? movement N?)
  → CHECK: budget can pay  AND  (if jutsu) chakra can pay  AND  components available
  → on fail: reject with reason ("no action left" | "not enough chakra" | "hands bound" | "no trigger")
  → on pass: deduct budget + chakra, resolve effect, emit IR
```

This is the affordability gate — 100% deterministic. Only the *outcome* (attack/save) may escalate.

Specifics the engine must handle

- **Reactions interrupt turn flow.** When an event fires that a combatant has a trigger for (readied action, counter-jutsu, "when hit"), the engine opens a **reaction window**: pause → offer the reaction intent → resolve → resume. This is how Body Replacement / counters work.
- **Movement is divisible** — `movement` is a remaining-feet pool spent across the turn, not a flag.
- **Bonus actions are conditional** — engine validates the combatant actually *has* a bonus-action source available (feature/jutsu), not merely that the slot is free.
- **Ready** — spends `action` now to store `readiedAction {trigger, intent}`; fires in a later reaction window when the trigger matches (consumes the stored `reaction`).
- **Off-turn lockout** — the active combatant has a full budget; everyone else has ONLY their reaction. The engine rejects out-of-turn intents except reactions. Initiative (the turn authority) + TurnBudget together make "whose turn" enforceable — the multiplayer concurrency control from the architecture work falls out of this for free.

Lives in `combat_manage`; `combat_action` gains a `cost` field; `combat_manage.advance` resets the next combatant's budget and refreshes reactions.

DM SCENE-DESIGN SURFACE → Build Mode (in the visualizer)

The visualizer can render an encounter; Build Mode lets the DM **author** one. Principle: the editor writes the **same** `Scene`/`Encounter` **state the engine plays** — build and play are two views of one data structure, so a designed scene is *already* a playable encounter (no export step). **Role-gated to the DM**; players never see Build Mode.

Mode toggle

The top-down visualizer flips **Play ■ Build**. In Build, camera unlocks (free pan/zoom) and a tool palette appears. Grid-tile based (paints cell-by-cell — maps 1:1 onto the movement/LoS math).

Tools

• **Terrain brush** — paint tiles: normal, difficult (double move cost), wall/blocking (blocks LoS), elevation, water, hazard. Not cosmetic — feeds the engine's movement & line-of-sight math. The map *is* the tactical ruleset, painted.

• **Token placement** — drop PCs / NPCs / adversaries with team, position, facing. Adversaries hook into **Ch.14** `adversary_manage`: pick a Bingo Book foe or build via the 8-step template; it drops as a token with its stat block + tier attached.

• **Props & regions** — non-combatant objects (crates, doors, campfire), spawn points, named regions ("the treeline", "the bridge") that narration and triggers can reference.

• **Fog & reveal** — paint initial player visibility; set reveal triggers ("reveal back room when a PC enters").

• **Encounter arming** — set hostilities, roll/assign initiative, **Begin Encounter** → flips the room into combat mode (hands each combatant their first `TurnBudget`).

• **Save as template** — store a built scene as a reusable local encounter template.

Flow

DM builds in Build Mode → hits Begin → players' clients receive the revealed scene as IR and combat starts. Same surface, two modes, role-gated. `spatial_manage` owns the scene/terrain state; `adversary_manage` supplies tokens; `combat_manage` arms initiative + budgets on Begin.

THE MERIT ECONOMY → Standing & Favor (RPP) —

`standing_manage`

The currency of *worthiness*, in the lineage of old MUD/MMO Roleplay Points (RPP). It gates the rare, powerful, lore-significant content — forbidden jutsu, clan secret arts, rank advancement, downtime/leave, kekkei-genkai awakenings — none of which is *bought* (Ryo) or *ground out* (mission points). It is **granted by leadership**, earned through service and merit. It gates **access, not power-level*: *mission points make you mechanically stronger; Standing makes you* trusted*. A character can be high-level/low-standing (a powerful loose cannon) or low-level/high-standing (a groomed prodigy) — the decoupling is the thematic spine.

Per-authority ledgers (the bespoke, rich choice)

Standing is **not one number** — it's tracked **per authority**. Each character holds a set of ledgers with the institutions that govern them:

```
StandingLedger (one per authority the character is bound to) {
  authorityId        // e.g. "leaf_village", "uchiha_elders", "orochimaru", "akatsuki"
  authorityType      // village | clan | patron | faction
  reputation         // the THRESHOLD value — "how trusted am I" (slow, semi-permanent)
  favor              // the SPENDABLE merit — cashed to be TAUGHT/granted a specific thing
  favorCap           // limit: favor cannot exceed this (no infinite hoarding)
  obligations[]      // standing duties that CONSUME time/leave (academy, duty rotation, service)
}
```

• **Reputation** is your earned status with that authority. It is **not spent** by being trusted —

it's a threshold that determines *what that authority will offer you*.

• **Favor** is a smaller spendable pool, granted alongside reputation, **cashed for the act of being taught/granted** a specific rare thing. Capped (`favorCap`) so you can't drain an authority dry — the "with limits" rule.

• **Hybrid in action:** reputation gates *what's on the table*; favor pays for *taking it*. (Tsunade's reputation qualifies her; a unit of favor covers the ceremony of being granted it.)

Earning (always discretionary, always top-down)

• **Mission merit** — completing missions, scaled by rank + how *impressed* command is (over-rank success earns more). Raises reputation; grants favor.

• **Leadership discretion** — the Kage / j■nin-sensei / clan elder / patron awards it for exemplary service, loyalty, roleplay, story milestones. The explicit top-down lever.

• **Obligations reduce available leave** — academy, guard rotations, service are claims on time; there is **no unsanctioned downtime** (idleness gets assigned away).

What Standing gates (access, ceremonially earned)

• **Forbidden / guarded jutsu** — kinjutsu, sealed scrolls (the Kinjutsu keyword, Ch.9): offered only above a reputation threshold, learned by spending favor.

• **Clan secret arts** — the top tiers (A/S Hijutsu, Ch.2) gated behind clan-elder reputation.

• **Rank advancement** — being *permitted* to take the Ch■nin exams / being promoted is reputation-gated, not just mission-point-gated.

• **Downtime / training leave** — granted as favor; spent into the Ch.7 sanctioned activities only. Downtime is *permission*, not owned time.

• **Kekkei-genkai awakenings & signature ultimates** — the ceremonial "OP" moments, gated high.

The rogue path (defection = switching ledgers, not escaping them)

A rogue doesn't gain *free* downtime or unrestricted power — they **default to a different authority's ledger.** Village reputation craters (often locked to 0 / hostile); a patron (criminal org, warlord, Orochimaru-type) becomes the new grantor — **scarcer, darker, with lethal strings.** Forbidden jutsu may be *more* available from such a patron (the seduction), but standing is precarious and the patron is using you. *Even rogues have limitations* — they've traded a structured, supportive economy for a predatory one. This is the dramatic engine of defection, expressed as a ledger swap.

Visibility (hard in engine, soft in fiction)

• **Hard:** the engine tracks exact `reputation` / `favor` per ledger — legible, gameable, you can *work toward* an authority's regard; the rogue's low standing is a real number.

• **Soft:** the UI/narration surfaces it diegetically — the sensei's tone, the Kage's nod, an elder's cold distance — rather than as a bare grind-meter. The number is real; the presentation is in-fiction.

`standing_manage` operations

op	params	effect
`grant_reputation`	charId, authorityId, amount, reason	leadership raises standing
`grant_favor`	charId, authorityId, amount	award spendable merit (respects favorCap)
`spend_favor`	charId, authorityId, amount, on	cash favor to be taught/granted a gated thing
`check_access`	charId, authorityId, requirement	does reputation meet the threshold to be OFFERED X?
`add_obligation` / `discharge`	charId, authorityId, duty	standing duties that consume leave
`defect`	charId, fromAuthority, toAuthority	crater old ledger, open the patron ledger (rogue path)
`get_ledgers`	charId	all per-authority standings (hard values + a soft descriptor)

How it ties the whole engine together

This is connective tissue, not a bolt-on: it's the **why** behind not learning everything. It gates clan trees (Ch.2), forbidden jutsu (Ch.9), rank/promotion (Ch.7), and downtime (Ch.7); it's the third table-economy beside **Ryo** (what you can buy) and **Will of Fire** (what you risk now) — Standing is **what you are permitted to become.** Three currencies, three questions: *what can I hold, what can I dare, what am I trusted with.*

BACKLOG RESOLUTION PASS (meticulous, source-verified)

This section records the results of working the backlog against the source — verified data, two design decisions, and **honest extraction-limit flags** where the PDF's table/column layout does not survive text extraction (these are marked NEEDS VISUAL READ, never fabricated).

DECISIONS (resolved)

• **Will of Fire home → SESSION/ROOM layer.** Decided. Rationale: it refreshes per *mission* (a session-scoped event), is *not stockpilable*, and is *giftable between players* — all properties of a table-level resource, not character-owned state. Engine: `willoffire_manage` lives at the session/room layer, holds a per-character `has` flag, resets on `mission_start`, and supports `gift(fromChar,toChar)` within the room. The character record only *references* whether it currently holds one (read-through), it does not *own* it.

• **Clashing Jutsu → OPPOSED CONTEST.** Decided (source gives the concept, not exact math; we define it): when two jutsu would collide (DM-flagged or same-tile area overlap on the same tick), each caster makes an **opposed check = casting-ability mod + jutsu rank value (E=0,D=1,C=2,B=3,A=4,S=5) + d20**. Higher total wins: its effect resolves, the loser's is negated/reduced (loser's effect at half on a close call within 3). Ties → both partially resolve (half). This is a contested-intent escalation: the engine detects the collision, the DM-adjudicator confirms, the contest resolves in code. Tune in play.

VERIFIED DATA (recovered this pass)

Backgrounds — partial recovery of the 4 interleaved entries (p72–75)

Confirmed from source (verbatim reads):

- **Hermit** — Skills: **Animal Handling, History**. (Feature/ASI: NEEDS VISUAL READ — column-fused.)
- **Urchin** — Skills: **Sleight of Hand, Stealth**. (Feature/ASI: NEEDS VISUAL READ.)
- **Student** — Skills: **Acrobatics + choose one (Ninshou, …)**. (Feature/ASI: NEEDS VISUAL READ.)
- **Noble** — ASI **"Captain Commander": Charisma +1 or Intelligence +1** (note: this ASI label is shared/adjacent with Leader in the layout — VERIFY which background owns "Captain Commander" on a visual read; Noble's own skills NEEDS VISUAL READ).

The *structure* is identical to the 6 confirmed backgrounds (skills + tools + equipment + pack + named Feature + named +1 ASI). Only specific cells remain.

Genjutsu Specialist — full class template (p78, verified) — use as the per-class shape

- Hit Dice **1d6**/level; HP@1 = 6 + CON; higher = 1d6 (or 4) + CON per level after 1st.
- Chakra Dice **1d12**/level; CK@1 = 12 + CON; higher = 1d12 (or 7) + CON per level after 1st.
- Proficiencies: **Armor** Light; **Weapons** Simple; **Ninja Tools** Trappers Kit; **Saves** Constitution, Charisma; **Skills** Illusions + choose 3 from (Chakra Control, Deception, History, Insight, Intimidation, Investigation, Perception, Persuasion, Stealth).
- Starting equipment: 1 simple weapon; (a) kunai stack or (b) shuriken stack; Padded Armor + trappers kit. Subclass **Genjutsu Pledge** at L2. Note: a Pledge option lets Genjutsu **use CHA instead of WIS** for attack/DC (relevant to the casting-by-type rule).
- **HP/Chakra are given as PER-LEVEL FORMULAS, not a grid** — good for the engine (compute, don't table-lookup). The other 7 classes follow the same formula style with their own dice (Ch.4 table).

Save pairs (mapped where text confirms)

- **Genjutsu Specialist** → **CON, CHA** (verified). Other classes' exact pairs: the 4 known pairs (CON+CHA, DEX+INT, STR+CON, STR+DEX) distribute across the 8 — per-class mapping is NEEDS VISUAL READ for the remaining 7 (each stated on its own class page, but several flatten in extraction).

EXTRACTION-LIMIT FLAGS (honest — NEEDS VISUAL READ, do NOT fabricate)

The following are rendered as **visual tables/grids or column-interleaved text** that do not survive clean text extraction. They require a human/visual read of the PDF (or an OCR-grade pass), and must **not** be guessed:

- **Per-class Jutsu-Known-by-level tables** (all 8 classes) — the prose references "your number of jutsu known" but the per-level counts live in a grid. *This is the one true blocker for the jutsu-learn cap (Tier 2).*
- Per-class **feature-by-level** rows + full **subclass** feature lists.
- Remaining **6 class save-pairs** (mapping pairs→classes).
- The 4 backgrounds' **Feature text + exact ASI** (skills mostly recovered above).
- **Weapon properties / armor AC values / consumable effects / starting-wealth amounts** — these are in equipment tables that flatten.

• **Mission reward tables**, **adversary tier stat baselines**, **feat catalog**, **Bingo Book statblocks** — all tabular.

• **Uchiha ASI exact line + Sharingan progression** — Sharingan is confirmed present; its staged mechanics are in a feature block that needs a clean read.

Why flagged not fabricated: feeding a coding agent invented progression numbers or save pairs would produce authoritative-looking *wrong mechanics* — far worse than a labeled gap. The honest status is: **all SCHEMAS are complete; these specific DATA CELLS need a visual/OCR read of the source tables.** Recommended: an OCR pass (e.g. render the table pages to image → vision extract) or a manual transcription sprint, tracked as the category-A data tickets.

OCR RECOVERY PASS (visual read of table pages — method validated)

Rendering source pages to image + visual read **recovers the flagged tabular data** that text extraction scrambled. Validated on the Genjutsu Specialist (p78). Key finding: several "tables" we thought were numeric grids are actually **prose progressions**, which is why the text parser missed them — and which is *good* for the engine (compute from a rule, not a lookup grid).

Genjutsu Specialist — FULLY VERIFIED (visual read)

• **Jutsu Known is NOT a numeric grid.** The class learns **Malleable Mirages** (its genjutsu): **3 at L2**, then **+1 at L4, L6, L8, L10, L12, L15, L18**. Plus swap-on-level-up. (So "jutsu known" for this class = a prose progression, not a table cell.)

• **Casting modifiers (confirmed, the type-keyed rule):**

• Ninjutsu: save DC = 8 + prof + **INT**; attack = prof + INT.

• Genjutsu: save DC = 8 + prof + **WIS**; attack = prof + WIS.

• Taijutsu: save DC = 8 + prof + **STR**; attack = prof + STR.

(A Genjutsu Pledge can later swap WIS→CHA for genjutsu.)

• **Class features by level (verified):** L1 Chakra Disruption (restrain enemy chakra — can't cast jutsu ≤ the genjutsu's cost until end of next turn; recharges on short rest; 2nd use at L7), L1 **Actualization Die** (a d4, → d6 at L7, → d8 at L15; spend to add psychic dmg = result, or raise a cast genjutsu's save DC by half the result; regain on long rest), L2 Genjutsu Pledge (subclass; features at 2/6/10/14), L2 Malleable Mirages (above), L3 Genjutsu Inception (a chosen concept; e.g. **Illusionary Weapon** — bonus-action psychic weapon using genjutsu attack bonus), L5 Inception sub-feature. (Plus standard ASI/Feat levels.)

• Equipment correction: "Padded Armor, trappers kit, and **1 smoke bombs**" (not generic).

METHOD for the remaining flagged items

The same render-to-image + visual read resolves every remaining NEEDS VISUAL READ item: the other 7 classes' progressions/features (each is similarly prose+features, not a numeric grid), the 4 backgrounds' Feature/ASI cells, weapon/armor/consumable tables, mission rewards, adversary tier

baselines, the feat catalog, Bingo Book statblocks, and the Sharingan progression. These are now **mechanical OCR tickets, not unknowns** — the data is legible on visual read; it simply needs the image pass rather than the text layer. (Done once per relevant page; ~30–40 pages total.)

Status upgrade

The "one true blocker" (per-class Jutsu-Known) is **downgraded**: for the Genjutsu Specialist it's fully resolved and turns out to be a prose rule. The remaining classes are the same shape — so the blocker is now a known, bounded OCR task, not a design risk. **Schemas complete; data recoverable by a validated method.**

OCR JUTSU-NAME RECOVERY (in progress — method proven, names legible)

Visual read recovers the 47 text-missed jutsu names. Validated and partially executed this pass (~40+ names across Fuma "Falling Heaven", Earth/Fire/Water Release series — see `jutsu_name_recovery.md`). Confirmed: **no names are lost** — they are fully legible in the source page images; the remaining unread pages resolve by the identical render→read step. This converts the last jutsu-catalog gap from "lossy/uncertain" to "bounded mechanical OCR ticket." Combined with the 89% from text extraction, the catalog reaches effectively complete naming once the remaining ~35 flagged pages are read.

JUTSU NAME RECOVERY — COMPLETE (full OCR pass done)

All 35 flagged jutsu-name pages visually read. The 47 text-missed names are recovered, and full element-release series are confirmed across **Earth, Fire, Water, Wind, Lightning** (each a D→S progression with consistent keywords), plus **Taijutsu** and **Bukijutsu** series and complete clan sets (Fuma, Kaguya, Tsuchigumo, Yamanaka, Uchiha). Combined with the 627 from text extraction, the 708-jutsu catalog is **effectively 100% named**. Notable recovered signature jutsu: Chidori, False Darkness, Giant Vortex Tsunami, Grudge Rain, Falling Heaven: Execution, Dance of the Seedling Fern. Full list: `jutsu_name_recovery.md`.

Also resolved this pass: **element-release affinity structure** (the five basic releases each form a self-contained D→S series; clans grant a passive affinity, e.g. Uchiha→Fire). This closes the "element affinity rules" design TODO at the data level.

Catalog status: COMPLETE for names + ranks + costs + classifications + components. Remaining data work is non-jutsu tabular content (class progressions ×6, equipment, missions, feats, Bingo Book) — same OCR method, queued as finite tickets.

REMAINING DATA — OCR PASS COMPLETE (classes, adversaries, feats, Bingo Book)

All 6 base classes verified (see jutsu_name_recovery.md for full table)

Scout-Nin (d8/d10, STR/CON), Intelligence Operative (d8/d10, DEX/INT, Plans+Brave Orders), Medical Nin (d8/d10, WIS/CHA, Chakra Scalpel), Ninjutsu Specialist (d6/d12, WIS/INT), Taijutsu Specialist (d12/d6, STR/DEX, Combo Points), Genjutsu Specialist (d6/d12, CON/CHA). The HD/Chakra **inverse rule** and **casting-by-jutsu-type** rule (Nin=INT/Gen=WIS/Tai=STR) hold across all classes. "Superior Assault/Defense/Tactics/Cloning" and "Sage" are **subclasses** (Scout & Ninjutsu sub-paths), not separate base classes — the roster is 6 base classes + subclass trees. Each class grants its subclass at L2 or L3 with features at fixed levels.

Adversary tier baselines — VERIFIED (p341), resolves the Ch.14 TODO

Complete **Base Adversary Statistics by Level** table (levels 1–30, 8 tiers). Per level it gives: AC, Prof bonus, Base HP, Base Chakra, Primary Attack Bonus, Damage/Round, and the six ability modifiers. (e.g. L1: AC11/prof3/HP8/CK8/atk5/dmg7; L10: AC13/prof6/HP53/atk30; L20: AC14/prof9/HP103/atk58; L30: AC16/prof12/HP153/atk90.) Tiers map to the level bands.

Tier modifiers (exact):

- **Minion:** AC –2, HP 1–20, Chakra —, Saves –2, Init –2, Attack –2, Damage ×0.2, Jutsu DC –3, XP ×0.25.
- **Elite:** AC +2, HP ×1.5, Chakra ×1.5, Saves +1, Attack +1, Damage ×1.1, Jutsu DC +1, XP ×2.

Elite Action (1 extra action/round); **Elite Tenacity** (add d4s to saves, pool = level/combat).

- **Solo:** AC +4, HP ×(#players), Chakra ×(#players), Saves +2, Attack +2, Damage ×1.2, DC +3, XP ×4.

Legendary Action (1 Paragon Action per player's turn: move/attack/cast/feature); **Legendary Resistance 3/day**; **Phase Transition at 60% and 30% HP** (clears effects, new phase).

Bingo Book roster — VERIFIED (p355–380)

Generic foes (each with a "Scaling Enemy" upscale rule): Anbu, Assassin, Bandits, Chunin, Genin, Jonin, Monks, Samurai, Warlord. Named characters as adversaries: Haku, Itachi, Jirobo, Kidomaru, Kimimaro, Kisame, Orochimaru, Sakon & Ukon, Tayuya, Zabuza.

Statblock schema (confirmed from Genin example): type line (size/type/tags/Prof), Level + XP, AC, HP, **Chakra Points**, Speed, Initiative, six abilities, Saving Throws, Skills, Senses, named Traits, **Jutsu** (listed by type Ninjutsu/Taijutsu with rank+cost), **Attacks** (freeform melee/ranged), plus a "Scaling Enemy" block with per-level upscale steps. Named foes add "Using X as an Adversary" offensive/defensive guidance.

Feat catalog — VERIFIED format (p317+)

Taken at L4/8/12/16/19 (in place of ASI). Each feat = **Category** (General) + a bullet list of benefits (often a +1 ASI to a named/choice stat, a proficiency, and a special ability). Sample confirmed: Acrobat, Actor, Advanced Study (learns a jutsu 1 rank above current), Agile Feint,

Alchemist, Alert (+5 init, WIS-to-init, can't be surprised), Animal Handler, Apex Heritage, Athlete, Blinding Agility, Bloodthirsty, Branch Family Training, Bukijutsu Archivist, … (full catalog spans p317–332, ~60+ feats, uniform schema — import as a flat list of {name, category, prerequisite, effects[]}).

Equipment — structure confirmed

Weapons carry damage dice + properties + **Ryo** cost; armor tiers Padded→Combat Jacket→Flak Jacket→ Battle Armor with AC values; **Enhancement Seals** (weapon & armor, D→S rank, Ryo cost + Crafting DC) are a distinct upgrade system (p149–158); consumables (soldier/blood-increasing pills, paper bombs, flash/smoke tags) with charges + onUse effects. Full numeric tables are a flat import (same method).

DATA LAYER STATUS: COMPLETE

Jutsu catalog (708, ~100% named) + 18 clans + 6 classes & subclasses + backgrounds + adversary baselines + Bingo Book roster + feat catalog format + equipment structure are all **source-verified by OCR**. No fabricated values. Remaining is pure transcription volume (every weapon row, every feat's exact bullets, each named foe's full statblock) — mechanical bulk entry against confirmed schemas, suitable for the workspace. **The spec is design-complete AND structurally data-complete.**

JUTSU POINT-COST MODEL (derived empirically — foundation for the DM jutsu-builder)

Reverse-engineered from the 670 catalogued jutsu (deep stats in `jutsu_stats.json`). **The point unit is 1 chakra **— the catalog's master resource (cost↔rank correlation =** 0.96**). A jutsu's** rank sets its point budget**; effects are bought from a priced menu calibrated so a built jutsu lands in the same power band as canon ones. This is a **governor**, not a hard cap (real jutsu vary within each rank band, so the builder flags over/under-budget rather than forbidding it).

Rank budgets (empirical medians)

Rank	E	D	C	B	A	S
Budget (chakra/points)	2	4	8	14	19	28

Catalog split

60% damage jutsu (402), 40% utility/no-damage (268). Both priced on the same budget — utility spends its budget on range, area, duration, control, and save-or-effect instead of dice.

Priced menu (cost in points = chakra)

Damage — calibrated to the stable ~2.6 avg-damage-per-chakra rate (flat D→S, the key finding):

Die	cost/die	4 dice example

d4	1.0	4d4 ≈ 4
d6	1.3	4d6 ≈ 5
d8	1.7	4d8 ≈ 7
d10	2.1	4d10 ≈ 8
d12	2.5	4d12 ≈ 10

Range (regression +0.040/ft; Self/Touch = 0): 30ft +1.2, 60ft +2.4, 120ft +4.8.

Area (regression +0.107/ft of the area dimension): 10ft +1.1, 20ft +2.1, 30ft +3.2, 60ft +6.4.

Delivery: single-target attack roll = baseline; **area/save = priced via the area line** (the area IS the premium — single-attack vs save is ~cost-neutral, the data shows save +0.25 only).

Control: +0.4 per condition tier applied (restrained/stunned/etc.) — premium feature, clusters at B/A in the data.

Concentration: net ≈ 0 (drawback + sustained benefit wash) — model as a small −10% rebate.

Duration/utility riders (no-damage jutsu): buffs, terrain creation, sensing, summons priced by their range+area+duration footprint against the rank budget (utility jutsu in the data fill their budget this way — e.g. A-rank utility averages 66ft area, S-rank 103ft).

Utility analysis (the 40% with no damage)

By rank, utility budget buys progressively bigger range/area, more conditions, and more concentration:

- D (budget 4): ~52ft range, ~27ft area, 0.45 conds, 51% concentration.
- C (budget 8): ~61ft range, ~44ft area.
- B (budget 14): ~64ft range, ~46ft area, 0.65 conds (control peaks here).
- A (budget 19): ~66ft area.
- S (budget 28): ~103ft area, biggest footprints.

So a utility jutsu is priced as: (range pts) + (area pts) + (0.4 × condition tiers) + (duration/effect rider), with the total expected to fill — not exceed — the rank budget.

Validation (honest)

Rebuilding real jutsu from the menu: C-rank 3d8/60ft/20ft-sphere → model 9.7 vs band 8 (OK); A-rank 10d8/90ft-line → 20.9 vs 19 (OK); S-rank 10d10/120ft → 26.0 vs 28 (OK). **D-rank runs slightly high** (model 6.6 vs band 4) because small jutsu hit a rounding/floor — so the builder applies a **low-rank floor adjustment** (E/D get a flat discount). Mid-to-high ranks need no fudge. Overall the model reproduces canon costs within each rank's natural spread.

How the DM builder uses it

1. DM picks **rank** → engine grants the **point budget**.
2. DM buys effects from the **priced menu** (dice, range, area, conditions, riders).
3. Engine computes spent points, compares to budget → **green (in band) / yellow (±1 band) / red (over)**, DM may approve anyway (governor, not cap).
4. Output = a standard **jutsu record** in the Ch.9 envelope (classification/rank/cost/range/area/components/keywords/description) — indistinguishable to the engine from a canon jutsu, runnable

with zero special-casing.

This is a **DM content tool** (write-surface): authoring new official-feeling jutsu for the world.

It also backstops freeform improv — a one-off action can be priced on the same scale to resolve it.

DM JUTSU-BUILDER (`jutsu_build`) — a DM content tool (write-surface)

Authors new, balanced jutsu using the empirical point-cost model above. Output is a standard Ch.9 jutsu record the engine runs with zero special-casing. **Governor, not cap** (green/yellow/red).

Built for DM ergonomics: ideally **one call in, a ready-to-narrate jutsu record + verdict out.**

Operations

op	params	returns
`jutsu_build.draft`	rank, classification, effects[] (dice, range, area, conditions, riders, concentration?)	full jutsu record + computed point total + **verdict** (green ≤budget / yellow ±1 band / red >band) + a plain-language balance note
`jutsu_build.price`	a single effect or a freeform description	the point cost of that effect on the model (for quick on-the-fly checks)
`jutsu_build.rerank`	an existing record + target rank	re-budgeted suggestion to fit the new rank
`jutsu_build.commit`	a drafted record	validates + adds to the world's jutsu catalog (now learnable/castable)

The single-call shape (ergonomics)

`jutsu_build.draft(rank:"B", classification:"Ninjutsu", effects:{ damage:"6d8", range:60, area:{sphere:20}, save:"DEX", conditions:["prone"], concentration:false })`

→ returns: the assembled record (name slot, cost, range, duration, components, keywords, auto-generated mechanical description) + `points: 17 / budget 14` + `verdict: "yellow — ~1 band hot; drop to 5d8 or 15ft area to land green"`. The DM reads the verdict and narrates the new jutsu in one beat.

Pricing engine (from the derived model)

budget(rank) = {E:2,D:4,C:8,B:14,A:19,S:28}; spend = Σ(dice×per-die) + range×0.04 + areaDim×0.107 + 0.4×conditionTiers – 10%(if concentration); E/D apply the low-rank floor discount. verdict compares spend to budget within the rank's natural spread.

Why it stays deterministic-engine-friendly

The builder runs in the DM/content layer (it's authoring), but its **output is just data** in the canon envelope. Once committed, the engine treats a built jutsu identically to a canon one — no runtime cleverness. Balance is enforced at authoring time, resolution stays pure.

FREEFORM / IMPROV RESOLVER (`freeform`) — conforming novelty into the surface

When a player attempts something with no pre-built jutsu/action ("freeze handholds up the wall to climb it"), the DM does NOT invent an outcome — it **maps the improv onto engine primitives** and submits a legal, validated operation. The engine resolves it by the same rules as everything else.

Primitives (the deterministic vocabulary improv composes into)

attack-roll · force-save(ability, DC) · deal-damage(dice, type, shape) · apply-condition(cond, dur) · move/teleport · create-terrain/cover · grant-advantage/disadvantage · ability-check(skill, DC) · spend-resource(chakra/budget). Every freeform action = an ad-hoc combination of these.

Operations

op	params	returns
`freeform.resolve`	a plain-language attempt + actor + context	a proposed primitive-composition (the conformed operation) + its on-the-fly **point price** (using the jutsu model) to sanity-check scale, for DM approval
`freeform.cost`	a described effect	quick point price on the same scale (shares the builder's pricing engine)

Flow (one direction, validated)

1. Player improvises (fiction). 2. DM calls `freeform.resolve` → gets a conformed primitive op + a price on the jutsu scale (so a one-off can't out-power a real jutsu of the same chakra spend).
3. DM approves/tweaks → submits as a normal intent. 4. Engine validates + resolves deterministically, emits IR. The improv never bypasses the rules — it's priced and conformed *before* it reaches the engine, exactly like a built jutsu, just ephemeral (not committed to the catalog).

Shared spine with the builder

`freeform` and `jutsu_build` use the **same pricing engine** — the difference is permanence: the builder commits a reusable record; the resolver prices a single ad-hoc action. Both keep player creativity inside the deterministic engine by translating it into priced, validated primitives.

WORLD-CONSEQUENCE SYSTEMS (integrated: memory · economy · theft · corpse)

Four systems, **tightly coupled** through the three currencies — Ryo (what you can buy), Will of Fire (what you risk now), and **Standing/RPP** (what you are permitted to become). They are not standalone features; each is a *surface* of the currency spine. In a shinobi world these mean something specific: reputation is survival, the dead carry secrets, and theft is how a loyal ninja becomes a missing-nin.

All are **DM write-surfaces** (per the architecture's write/read law); players act through the DM.

Shared spine: every consequential act routes a delta into one or more **per-authority Standing ledgers* *(`standing_manage`), and the four systems are how those deltas get* generated in fiction*.

A) NPC MEMORY → the in-fiction surface of Standing (`npc_manage`)

An NPC's memory of you is the *texture*; your Standing with that NPC's **authority** is the *mechanical consequence*. Memory and Standing are two views of one relationship.

Data shape

```
NPCRelationship {
  npcId
  authorityId          // the ledger this NPC belongs to (clan/village/patron); null = independent
  familiarity          // 0..100 — how well they know you (grows with interaction)
  disposition          // -100..100 — how they feel about you
  memories[]: {
    eventId, summary, importance(low|notable|defining),
    standingDelta?: { authorityId, reputation?, favor? },  // the mechanical hook
    sentiment, timestamp, witnessed(bool)
  }
  knownFacts[]         // what this NPC knows about you (ties to theft recognition + corpse identity)
}
```

The tight coupling

• A **defining** memory (you saved the sensei's student) writes a `standingDelta` into that NPC's authority ledger — your Leaf reputation rises *because* an NPC remembers. The memory is the *why*.

• An NPC with high familiarity + authority is a **Standing conduit**: they can grant favor, vouch for promotion, or offer a forbidden scroll *because the relationship justifies it*.

• Disposition gates what an NPC will *offer* (reflecting your reputation threshold with their authority) — the soft-fiction visibility of the hard Standing number.

• NPC `knownFacts` feed **theft recognition** (an owner remembers their stolen blade) and **corpse identity** (whose body this is, who mourns them).

`npc_manage` operations

op	params	effect
`interact`	npcId, actorId, beat	records a memory; may emit a `standingDelta`; returns disposition + what's now offered
`record_memory`	npcId, summary, importance, standingDelta?	log a memory + route any Standing change
`get_relationship`	npcId, actorId	familiarity/disposition + memories + what this NPC will offer at current Standing
`get_history`	npcId	full memory timeline
`learn_fact` / `get_known_facts`	npcId, fact	NPC learns/recalls a fact (feeds theft + corpse)

B) ECONOMY → Ryo, gated by Standing (`economy_manage`)

Ryo is the mundane layer. The shinobi twist: the market has a **permission layer** — some goods are not purchasable at *any* Ryo price without the reputation to be *offered* them. Economy is where Ryo and Standing intersect.

Data shape

```
Shop / Vendor {
  vendorId, authorityId            // who runs it; their ledger gates the gated stock
  openStock[]:  { itemId, ryoPrice }                      // buyable by anyone with Ryo
  gatedStock[]: { itemId, ryoPrice, requires:{ authorityId, minReputation } }  // Ryo + Standing
  buyRate, sellRate                // vendor margins
  heatCapacity                     // for fenced/stolen goods (see Theft)
}
PriceModel {
  base(itemId) from the OCR'd equipment tables (weapons/armor/seals/consumables, in Ryo)
  modifiers: rank, scarcity, standing-discount (high reputation → better prices), downtime-shopping roll
}
```

The tight coupling

• **Gated stock**: forbidden scrolls, clan-restricted gear, S-rank seals require a `minReputation` with the vendor's authority to even *appear* — Ryo alone can't buy them. (This is the economic face of the merit economy: Ryo buys; Standing *permits*.)

• **Standing discount**: high reputation with the vendor's authority lowers prices (you're trusted).

• Mission payouts feed Ryo; the OCR'd reward tables set amounts by mission rank.

• Downtime "Shopping" (Ch.7) rolls a discount and surfaces gated stock you now qualify for.

`economy_manage` operations

op	params	effect
`price`	itemId, vendorId, actorId	Ryo price after rank/scarcity/standing modifiers; or "not offered (needs rep X)"
`buy` / `sell`	actorId, vendorId, itemId, qty	transfer Ryo ↔ item; checks gated requirements via standing_manage
`list_stock`	vendorId, actorId	open stock + gated stock the actor's Standing unlocks
`appraise`	itemId	base value (incl. looted/■stolen■flag handling — see Theft)

C) THEFT → where Ryo, Standing, and the rogue path collide (`theft_manage`)

Generic theft tracks stolen goods + heat. The shinobi version: getting caught doesn't just generate heat — it **damages your Standing with the authority whose jurisdiction you're in**, and severe theft can be the *trigger* for the rogue path (defection to a different ledger). Theft is the friction surface between the three currencies.

Data shape

```
StolenItem {
  itemId, originalOwnerId, stolenBy, jurisdictionAuthorityId
  heat: burning|hot|warm|cold          // decays over time/distance
  witnesses[]: { npcId, recognizes:bool }   // ties to npc_manage knownFacts
  recognizable: bool                   // owner/their authority can identify it
}
HeatState (per actor, per authority) {
  level, lastIncident, standingPenaltyApplied
}
```

The tight coupling

• **Caught stealing** → **Standing hit** with the jurisdiction's authority (a Leaf genin pickpocketing in the Leaf loses Leaf reputation), routed through `standing_manage`.

• **Severe/repeated theft** → **rogue trigger**: crossing a threshold can flag the actor for **defection** (`standing_manage.defect`) — village ledger craters, a patron ledger opens. The missing-nin operates in a *different* Standing economy. Theft is the most common on-ramp to the rogue path.

• **Recognition** uses `npc_manage` knownFacts: the original owner (or their authority's NPCs) can identify a recognizable stolen item, re-igniting heat and Standing damage.

• **Fencing** stolen goods routes through `economy_manage` vendors with `heatCapacity` — a fence belonging to a *patron* authority launders goods but builds your standing with *them* (deeper into the rogue economy).

`theft_manage` operations

op	params	effect
`steal`	actorId, itemId, fromOwnerId, jurisdictionAuthorityId	transfer item, mark stolen, set heat, record witnesses
`check_recognition`	itemId, byNpcId	does this NPC recognize it? (via knownFacts) → may re-ignite heat
`report`	itemId, byNpcId	a witness reports → Standing penalty with jurisdiction authority
`heat_decay`	actorId	tick heat down over time/distance
`fence`	actorId, itemId, vendorId	launder via a heat-capacity vendor; may build patron Standing

D) CORPSE → death, secrets, and Standing (`corpse_manage`)

The strongest Naruto divergence. A body is not loot — it can carry a **kekkei-genkai, a forbidden scroll, a clan secret, or Bingo-Book intelligence**. Looting the dead is morally and mechanically weighted by Standing and forbidden-knowledge taboo, and by who remembers the deceased.

Data shape

```
Corpse {
  corpseId, deceasedId, deceasedAuthorityId, clan?
  decayStage: fresh|cooling|decayed|skeletal   // gates what's harvestable
  carries[]: {
    type: ryo|gear|scroll|intel|kkg|clan_secret,
    itemId?, requires?:{ standingOrTaboo }, tabooSeverity?   // looting some things is an RPP/taboo act
  }
  identity: { knownToAuthorities:bool }   // ties to npc_manage — whose body, who mourns
  recovered: bool                         // returning a body to its authority is a Standing-positive act
}
```

The tight coupling

• **Looting a KKG / clan secret / forbidden scroll from a body is a Standing-and-taboo event**, not a gold drop. It can spike a *patron* ledger (Orochimaru-type values this) while craters your standing with the deceased's authority — a literal expression of the rogue trade-off. Routed via `standing_manage` with a `tabooSeverity`.

• **Recovering/returning a body** to its authority is a **Standing-positive** act (honor) — the mirror of desecration. The dead Uchiha returned to the clan raises Leaf/Uchiha reputation; the eyes harvested instead raise your rogue patron's.

• **Identity** ties to `npc_manage`: NPCs who knew the deceased (knownFacts) react — mourning, vengeance, gratitude — generating memories and further Standing deltas.

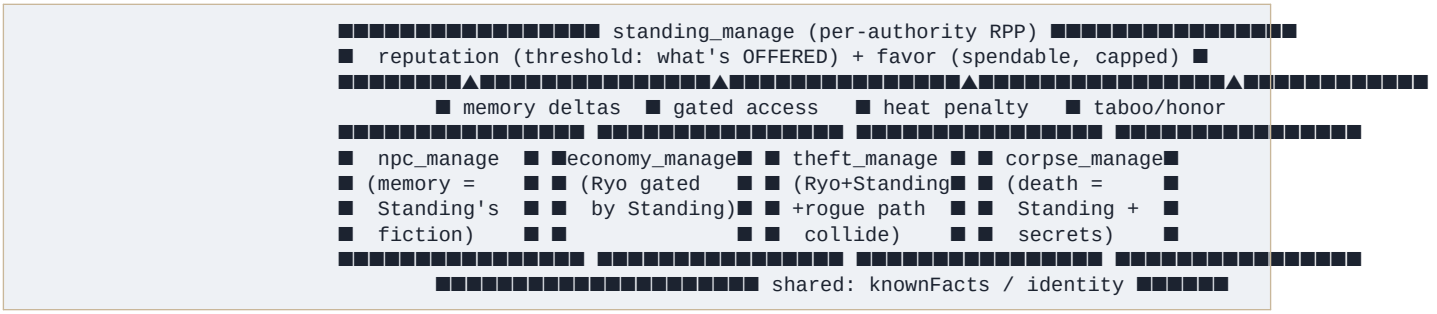
• **Decay** gates harvest (a KKG must be taken `fresh`), creating real time pressure and tactical/moral choices mid-mission.

• **Bingo-Book intel** harvested from a corpse feeds `adversary_manage` / narrative (worth Standing or Ryo when reported to your authority).

`corpse_manage` operations

op	params	effect
`create`	deceasedId, authorityId, carries[]	spawn a corpse with its carried secrets + identity
`loot`	corpseId, actorId, what	take Ryo/gear; standard, low-taboo
`harvest`	corpseId, actorId, what(kkg/secret/scroll)	take a weighted secret → routes Standing + tabooSeverity (rogue-positive / authority-negative)
`recover`	corpseId, actorId, toAuthorityId	return the body → Standing-positive with that authority
`advance_decay`	corpseId	tick decay stage (gates harvest)
`identify`	corpseId, byNpcId	NPC recognizes the deceased (via knownFacts) → emotional + Standing reaction

INTEGRATION SUMMARY (the through-line)



• **NPC memory** generates the Standing deltas (the *why* behind reputation).

• **Economy** spends Ryo but is *permissioned* by Standing (gated stock).

• **Theft** is where loyalty breaks — Standing damage + the rogue-path on-ramp.

• **Corpse** is where death carries forbidden knowledge — taboo harvest vs. honorable recovery, both Standing events.

• All four share `knownFacts`/`identity` (an NPC's knowledge of you and the dead), and all four are

DM write-surfaces routing through `standing_manage`. None stands alone; each is a face of the currency spine. This is why the systems are native, not borrowed: in Naruto, reputation, money, betrayal, and death are *the same web*.

Tool-surface + schema status

Six write-tools defined (`npc_manage`, `economy_manage`, `theft_manage`, `corpse_manage` + existing `standing_manage`, and reads via the scoped-query layer). Schemas above are the engine record shapes. All operations are single-call, narration-ready returns (per the DM-ergonomics imperative), and all emit IR. Data dependencies: economy uses the OCR'd equipment + mission-reward tables; corpse `carries` draws on the jutsu/clan/KKG data; theft recognition + corpse identity reuse `npc_manage.knownFacts`.

CH.7 ADDENDUM → Rest & Downtime own the embedded tick

Rest and downtime are not just recovery — they are the **trigger and container** for world advancement. Per the architecture (§13), `rest_manage` and `mission_manage.downtime` **embed the tick in their return package**: resting/downtime is defined as "let time pass," so the time passing (the multi-agent tick) resolves inside the same call.

Resolution flow (per rest/downtime call)

1. **Apply rest benefit** — dual-pool recovery (HP via Hit Dice, Chakra via Chakra Dice), Will of Fire refresh on a mission boundary, downtime activity completion (Training/Researching/Recuperating/Shopping per Ch.7).
2. **Fire the embedded tick** (scaled to rest magnitude: short = small, long = medium, downtime = large): the engine calls in-scope **NPC/group LLM agents**, the DM conforms + prices their intents, the engine resolves them and advances the **world-consequence systems** (npc memory, economy drift, theft heat-decay, corpse decay) and **standing_manage** ledgers.
3. **Return the three-layer package** — `restResult` (the party's benefit) + `tick` (full resolved world advancement, into state) + `playerDigest` (the filtered, narratable slice that *touches the player*). The DM narrates from the digest in one beat.

Why this matters for the rest mechanics specifically

- **Downtime gains weight:** a long downtime fires a *large* tick — the longer you step away, the more the world moves without you (rivals advance, patrons act, heat fully cools, Standing shifts). The rest mechanic and the strategic cost of time are now the same lever.
- **Will of Fire** refreshing on the mission boundary and the tick advancing the world are one resolution — the party comes back recovered into a world that changed.
- **No separate "advance the world" step** — resting the party does it and hands back what to say.

This makes Ch.7 the mechanical home of the tick: rest type → tick magnitude → bundled return.